VIDEO GENERATION SYSTEM

Complete Architecture, Workflow & Implementation Strategy

Automated Explanatory Video Pipeline for JEE-Level Questions

SYSTEM OVERVIEW

From a raw JEE question PDF → a fully rendered, narrated Manim video in under 2 minutes

Component	Primary Model	Fallback / Alt	Hardware Target
Brain (Solver)	Qwen2.5-VL-7B (fine-tuned)	Qwen2.5-VL-72B	DGX A100 / RTX 4080
Coder (Manim)	Qwen2.5-Coder-7B (fine-tuned)	DeepSeek-Coder-7B	DGX A100 / RTX 4080
TTS (Voice)	Kokoro-82M	MeloTTS (Hindi)	CPU (Windows PC)
Renderer	Manim CE + OpenGL	Manim CE + Cairo	RTX 4080 (GPU) / CPU
Embeddings (RAG)	BGE-M3	nomic-embed-text	CPU
Orchestrator	LangGraph	Custom DAG	DGX / Windows PC

1. The Core Idea

The fundamental goal is simple: a student pastes or uploads any JEE-level question, and within 2 minutes receives a high-quality explanatory video with Manim animations and a natural-sounding narration voice, exactly like a private teacher explaining it on a whiteboard.

The key insight driving every architectural decision in this system is:

> **80% of JEE visual explanations fall into ~12 well-defined animation patterns.**
>
> **The bottleneck is not the LLM — it is LaTeX compilation + render time + cold starts.**
>
> **A fine-tuned 7B model that knows JEE-specific Manim code beats a generic 72B model every time.**

Every optimization in the pipeline targets one of those three facts. The result is a system that produces 3Blue1Brown-quality animations for JEE content, running on hardware you already have, at a cost approaching zero.

---

2. Hardware Strategy & Modularity

The system is designed to run on two possible hardware configurations. Switching between them is a single environment variable change — the pipeline logic is identical; only the inference endpoint changes.

2.1 Available Hardware

DGX (Remote)	8 x NVIDIA A100 32GB SXM | 256GB total VRAM | CUDA 12.5 | Shared cluster — request 1-2 GPU allocation from lab admin
Windows PC (Local)	RTX 4080 16GB | 64GB RAM | Primary Manim render machine | Always-on, dedicated, no sharing conflicts

2.2 Role Assignment

These two machines are not interchangeable for all tasks — each has a natural role:

> **DGX A100:** LLM inference (Brain + Director + Coder via vLLM) and all fine-tuning jobs. A100s have 2TB/s memory bandwidth vs RTX 4080's 700GB/s — they are materially faster for transformer inference.
>
> **RTX 4080 PC:** All Manim OpenGL rendering, TTS generation (Kokoro), Redis job queue, FastAPI orchestrator, FFmpeg assembly. OpenGL rendering is GPU rasterization — the RTX 4080 is ideal for this and it doesn't compete with LLM workloads.

2.3 Fallback Mode (RTX 4080 Only)

When DGX access is unavailable (maintenance, quota exhausted), the system automatically falls back to running both models on the RTX 4080 16GB using 4-bit AWQ quantization:

Qwen2.5-VL-7B AWQ	~8GB VRAM — fits with room to spare
Qwen2.5-Coder-7B AWQ	~8GB VRAM — total both models: ~16GB, right at limit
Render workers	Reduced from 6 to 3 parallel workers on RTX 4080
Latency impact	+15-25 seconds on inference, otherwise identical quality

The config switch:

```
INFERENCE_BACKEND=dgx   # or: INFERENCE_BACKEND=local
# All LLM client calls route through a backend abstraction layer
# that reads this variable and points to DGX:8001 or localhost:8001
```

---

3. Full Pipeline: Step-by-Step Workflow

The following describes exactly what happens from the moment a user submits a question to the moment the video is ready. Every step, every decision point, every model call is described in sequence.

3.1 Phase 0 — Input Ingestion & Cache Check

0	Input Ingestion, Hash & Cache Lookup

The user submits either: (a) a text question typed directly, (b) a screenshot or PDF page of a printed question. Both arrive at the FastAPI endpoint on the Windows PC.

Before any model is called, the system computes a deterministic hash of the input:

- For text: SHA-256 of the normalized question string (lowercase, whitespace stripped)
- For images: perceptual hash (pHash) using the imagehash library — this handles slight scan quality differences of the same question

This hash is checked against Redis. Cache hit means the video was already generated — return the CDN URL immediately, zero pipeline cost, sub-second response. Cache miss means create a job entry in PostgreSQL with status QUEUED and push the job ID to a Celery task queue. Return the job ID to the client with a polling URL.

Why a queue even at small scale: Manim render is CPU+GPU heavy. Even with 10 users, if 3 submit simultaneously, without a queue all 3 saturate the RTX 4080 and everyone gets 3x slower. The queue serializes render jobs intelligently.

3.2 Phase 1 — The Brain: Multimodal Solver & Scene Scriptwriter

1	Qwen2.5-VL-7B (fine-tuned) — Solve, Narrate, Structure

Model: Qwen2.5-VL-7B-Instruct (fine-tuned with LoRA on your JEE dataset)

Input: The question as text OR as a base64-encoded image (the model handles both natively)

Output: A strict JSON object containing an array of Scene objects

This is the most important model call in the entire pipeline. Everything downstream depends on the quality of this output. The model must simultaneously: understand the question (including reading printed text and diagrams from images), solve the mathematics correctly, write natural pedagogical narration a student would find helpful, and structure the explanation into discrete visual scenes.

The output schema that the Brain is trained to produce:

```
{ "question_type": "physics",
  "topic": "Kinematics - Projectile Motion",
  "difficulty": "medium",
  "scenes": [
    { "scene_id": "scene_01",
      "duration_estimate_sec": 7.5,
      "narration": "Let us set up our coordinate system. The ball is
projected..."),
      "visual_type": "projectile",
      "visual_params": { "u": 20, "theta": 60, "g": 10, "show_components":
true },
      "requires_codegen": false
    },
    { "scene_id": "scene_02", ... }
  ],
  "final_answer": "v = 17.3 m/s at 30 degrees below horizontal"
}
```

The visual_type field is constrained to a fixed enum of 12 values. This is critical — it prevents the model from inventing novel animation types that the downstream template system cannot handle. The Brain learns to classify the visual requirement into one of these categories:

equation_transform	Showing one equation morphing into another (TransformMatchingTex)
axes_plot	Graph on coordinate axes — position-time, velocity-time, energy diagrams
free_body	Force vectors on an object with labels
projectile	Parabolic arc with velocity component animation
circuit	Resistor/capacitor/inductor circuit with current flow animation
ray_diagram	Lens or mirror with incident and refracted rays
reaction_mechanism	Chemical reaction arrow-pushing, electron movement
orbital_diagram	Atomic orbital filling, hybridization diagrams
number_line	Points and intervals on a number line for inequalities
geometry_construction	Coordinate geometry — circles, parabolas, lines with labels
integration_area	Shaded area under/between curves for definite integrals
text_reveal	Simple step-by-step text appearing — for derivation steps

3.3 Phase 1.5 — Math Validator

1.5	Wolfram Alpha API — Deterministic Answer Verification

Every numerical answer and key equation in the Brain's output is verified against Wolfram Alpha. This is not an LLM call — it is a deterministic API call. If Wolfram disagrees with the Brain's final_answer, the orchestrator automatically retries the Brain call with a stronger model (either the 72B variant on DGX or a temperature-0 retry).

This catches the most embarrassing failure mode: a confidently wrong explanation video. For JEE content where numerical precision matters, this check costs almost nothing (Wolfram API is cheap) and prevents significant student trust damage.

3.4 Phase 2 — Parallel Fork: TTS + Director

2A+2B	Kokoro-82M (TTS) || Qwen2.5-VL-7B (Director) — Run in Parallel

After the Brain outputs the scene JSON, two completely independent processes start simultaneously using asyncio.gather():

Phase 2A: TTS — Kokoro-82M

Model: Kokoro-82M (Apache 2.0, runs on CPU, generates at 100x realtime)

What it does: Receives all narration strings from the Brain output. Submits all scenes to the TTS engine in a single batch. Returns audio files (.wav, one per scene) plus word-level timestamps.

The word-level timestamps are critical. They tell us exactly when in the audio each word is spoken. The Manim Coder uses these timestamps to place self.wait() calls so that animation events (equations appearing, arrows drawing) are synchronised to the exact moment the teacher's voice mentions them.

Kokoro at 82M parameters generates a 6-second audio clip in approximately 60 milliseconds. All scenes combined finish TTS in under 500ms total. This is essentially instant relative to the rest of the pipeline.

Phase 2B: Director — Qwen2.5-VL-7B (same model, different prompt)

Model: Qwen2.5-VL-7B (same vLLM instance as the Brain, different system prompt)

What it does: Receives the scene JSON from the Brain plus the estimated duration. Outputs precise Manim coordinate specifications.

The Director's job is narrower than it seems — because the Brain already chose a visual_type and visual_params, the Director only needs to decide layout details: where on screen should this equation appear, what color, how large, how to avoid overlap with other objects. It converts abstract descriptions into concrete Manim coordinates like UP*2 + LEFT*1.5.

Because it uses the same model and same vLLM server as the Brain, there is no extra GPU memory cost. The vLLM instance serves both calls from the same GPU allocation, and prefix caching means the system prompt tokens are computed once and reused.

3.5 Phase 3 — Template Matcher (Before Calling the Coder LLM)

3	Template Library — Pre-written Manim Code for Known Visual Patterns

This phase runs before any Coder LLM call and handles the majority of scenes. For each scene in the blueprint, the orchestrator checks whether the visual_type + visual_params combination can be fully handled by a pre-written, pre-tested Manim template.

A template is a fully parameterized Python class that produces a Manim Scene. It has been written, tested, and validated once by a human (you), and then used thousands of times without any LLM involvement:

```
class ProjectileTemplate(ManimTemplate):
    def can_handle(self, params: dict) -> bool:
        return all(k in params for k in ["u", "theta", "g"])

    def render(self, params: dict, audio_timestamps: dict) -> str:
        # Returns complete, valid Manim Scene Python code
        # Substitutes numeric values from params
        # Injects self.wait() at correct timestamp positions
        return self._template_str.format(**params, **audio_timestamps)
```

After building the full template library, you can expect:

Template coverage (Physics)	~75% of scenes — kinematics, optics, electricity, SHM, waves all heavily templated
Template coverage (Chemistry)	~60% — orbital diagrams, equilibrium tables, electrochemistry cells
Template coverage (Math)	~65% — integration areas, parabola/line intersections, determinant expansion
Overall coverage	~70% of all scenes served by templates, zero LLM Coder call needed
Template generation time	< 0.5 seconds (just string formatting + parameter injection)

For the remaining ~30% of scenes with novel visual requirements, the pipeline falls through to the Coder LLM.

3.6 Phase 4 — Coder LLM + Agentic Error Loop

4	Qwen2.5-Coder-7B (fine-tuned) + LangGraph Retry Loop + RAG

Model: Qwen2.5-Coder-7B-Instruct (fine-tuned on JEE-specific Manim code)

When called: Only for scenes where no template matches (~30% of scenes)

RAG Context Injection

Before generating code, the system retrieves relevant documentation chunks from ChromaDB. Two collections are maintained:

- Official Manim Community docs — scraped from docs.manim.community, chunked by class/method, embedded with BGE-M3
- Your own validated examples — every piece of Manim code that has successfully rendered in your system gets stored here. Over time this becomes the most useful part of the RAG system.

The query is constructed from the Director's blueprint text. Top-5 most similar chunks are injected into the Coder's context. This dramatically reduces hallucination of deprecated or non-existent Manim APIs.

The LangGraph Agentic Loop

```
[Generate Code] --> [Run in Docker Sandbox]
       ^                      |
       |              [Error? Parse traceback]
       |                      |
[Full LLM Regen] <-- [Known fix? Apply regex patch first]
                              |
                     [No match -> LLM retry with error context]
                              |
                      [Max 4 retries -> escalate to manual queue]
```

The known-fix patcher runs BEFORE any LLM retry. It checks the traceback against a library of common Manim API errors:

ShowCreation -> Create	Manim deprecated ShowCreation; auto-patched in 0ms
FadeInFrom -> FadeIn	API change in ManimCE 0.17+; auto-patched
GrowFromCenter -> GrowFromPoint	Auto-patched
Missing self.wait() at end	Auto-appended if render duration mismatch
LaTeX syntax errors	Common: missing $, unclosed {}, auto-detected and fixed

These known fixes handle ~30% of errors without an LLM call, saving both latency and cost. The library grows over time as new error patterns are discovered in production.

Docker Execution Sandbox

Generated Manim code is NEVER executed directly on the host. It runs inside a resource-limited Docker container:

- Memory limit: 4GB per container
- CPU limit: 2 cores per container
- Read-only filesystem except /output
- No network access (prevents any exfiltration from LLM-generated code)
- Destroyed after each scene — no state persists between jobs
- Timeout: 90 seconds (kills hung renders automatically)

Parallel Scene Processing

Scenes within a single question are independent of each other at render time. Up to 6 Docker containers run simultaneously on the Windows PC (3 if running in RTX-only fallback mode). For a typical 4-scene question:

Sequential (naive)	4 scenes x 25s = 100 seconds render time
Parallel (4 workers)	max(scene times) ≈ 25-30 seconds render time
Speedup	~3-4x on render time alone

3.7 Phase 5 — Quality Gate

5	Automated Validation Before Assembly

Every rendered scene .mp4 passes through automated checks before assembly. No LLM calls, purely deterministic:

Duration match	abs(rendered_duration - audio_duration) < 0.3s. If failed: feed back to Coder with wait() adjustment instruction
Resolution check	Must be exactly 1920x1080 or configured target
Blank frame detection	Extract 5 random frames with FFmpeg, check average brightness. All-black or all-white = render failure
File size sanity	A 6-second scene at 1080p: expected 2-15MB. Outside range = suspicious
Audio sync marker	Verify first frame has visual content (not a blank leader frame)

3.8 Phase 6 — Assembly & Streaming

6	FFmpeg Mux + Concat + HLS Streaming

Once all scenes pass the quality gate, FFmpeg handles the final assembly. Audio is baked into each scene video first (mux), then all scenes are concatenated:

```
# Step 1: Mux audio into each scene video
ffmpeg -i scene_01.mp4 -i scene_01.wav -c:v copy -c:a aac -shortest
scene_01_muxed.mp4

# Step 2: Concatenate all muxed scenes
ffmpeg -f concat -safe 0 -i file_list.txt -c copy final_output.mp4

# Step 3: Generate HLS playlist for streaming
ffmpeg -i final_output.mp4 -codec: copy -start_number 0 -hls_time 10 \
       -hls_list_size 0 -f hls output.m3u8
```

The HLS playlist is the key to perceived latency. As soon as scene_01_muxed.mp4 finishes the quality gate (~20-25 seconds into the pipeline), it is written to the HLS stream. The user's video player starts playing immediately. Scenes 2-4 finish and get appended to the stream as the user

watches scene 1. The subjective experience is: video starts in about 25 seconds, and it never buffers.

3.9 Phase 7 — Storage, Cache & Delivery

The completed video is uploaded to S3 (or Cloudflare R2 for cheaper egress). The S3 URL is written to PostgreSQL alongside the question hash. Redis is updated with the cache entry. The polling endpoint that the client has been checking now returns the video URL.

Cache TTL is 6 months. If the question hash is seen again within that window, the video is served immediately from CDN with zero pipeline cost. Given that JEE questions recur year to year and across coaching institutes, cache hit rates will grow significantly over time.

---

4. Latency Budget & Timeline

Here is the realistic end-to-end timing breakdown with all optimizations in place:

Phase	What Runs	P50 Time	P95 Time	Notes
0 - Cache Check	Redis lookup	< 0.1s	< 0.1s	Instant
1 - Brain	Qwen2.5-VL-7B (ft)	4-6s	8-12s	Image input ~2s slower
1.5 - Validator	Wolfram Alpha API	1-2s	3-5s	Parallel with Phase 2
2A - TTS	Kokoro-82M (all scenes)	0.3-0.5s	1s	Runs parallel with 2B
2B - Director	Same 7B, vLLM batch	2-4s	5-7s	Prefix cache hit
3 - Template Match	Python dict lookup	< 0.1s	0.2s	70% of scenes end here
4 - Coder (30% scenes)	Coder-7B (ft) + Docker	4-8s	15-25s	Novel scenes only
4 - LaTeX Cache	Disk lookup / compile	1-3s	5-10s	70% cache hit rate
4 - OpenGL Render	6 workers parallel	12-20s	20-35s	Dominant bottleneck
4 - Error Recovery	Known fix + LLM retry	0s	15-20s	Fine-tuned: ~15% retry
5 - Quality Gate	FFmpeg + Python	0.5s	1s	Per scene
6 - Assembly	FFmpeg mux + concat	3-5s	6-8s	
7 - Upload	S3/R2 upload	2-4s	5-7s	
TOTAL		30-50s	75-110s	HLS starts at ~25s

User-perceived latency: ~20-25 seconds to first frame of video playing (HLS stream begins

at scene 1 completion).

Total completion time P50: 30-50 seconds for most standard JEE questions.

Total completion time P95: 75-110 seconds for hardest questions with novel diagrams and one retry.

Maximum target: 2 minutes — well within budget at P99 except extreme edge cases.

---

5. Model Selection — Detailed Rationale

5.1 Brain + Director: Qwen2.5-VL-7B-Instruct

Why Qwen2.5-VL?	Only strong open-source model at 7B with native vision (reads printed equations and diagrams from images). Competes with GPT-4V on academic document understanding.
Why 7B not 72B?	At 7B with fine-tuning, inference is 4-6 seconds on A100 / 8-12s on RTX 4080. At 72B, it would be 30-60 seconds. The fine-tuning closes most of the quality gap for the specific task.
Why same model for Director?	Identical architecture = one GPU allocation serves both. vLLM prefix caching means the Director call (same system prompt prefix) gets a cache hit and runs faster.
License	Apache 2.0 — fully free for commercial use. No usage restrictions.
Quantization	AWQ 4-bit: 7B model fits in ~8GB VRAM. Full precision: ~14GB VRAM. Use AWQ on RTX 4080, full precision on A100 if available.

5.2 Coder: Qwen2.5-Coder-7B-Instruct

Why Coder variant?	Trained specifically on code generation tasks. Produces more syntactically valid Python with fewer import errors than the base VL model.
Why same Qwen family?	Single inference stack (one vLLM setup) serves all models. Shared tokenizer means no switching overhead. Easier fine-tuning with same LoRA hyperparameters.
Fine-tuning benefit	A fine-tuned Coder-7B that has seen 500 JEE Manim examples achieves ~85% first-pass render success vs ~45% for a generic model.
Alternative	DeepSeek-Coder-7B is a strong alternative if Qwen is unavailable. Similar performance profile.

5.3 TTS: Kokoro-82M

Why Kokoro?	82M parameters. Generates audio at 100x realtime speed. A 6-second clip in ~60ms. Apache 2.0. No API costs. Runs on CPU — does not consume GPU VRAM needed for LLMs.
Voice quality	Surprisingly natural for educational content. Not ElevenLabs-level expressiveness, but crisp, clear, and student-appropriate.

Hindi/Telugu support	MeloTTS is the recommended alternative for regional language support. Same runtime profile.
Word timestamps	Available via the timestamps=True parameter. Returns word-level start/end times that the Manim Coder uses for animation synchronization.

5.4 Embeddings: BGE-M3

Why BGE-M3?	Best-in-class open source embedding model. Supports hybrid retrieval (dense + sparse + colbert) in one model. Particularly strong on technical/scientific text.
Deployment	Runs on CPU, ~1.5GB RAM. No GPU needed. FastAPI microservice on Windows PC.
Alternatives	nomic-embed-text is lighter (smaller model, slightly lower quality) if memory is constrained.

---

6. Fine-Tuning Strategy

Fine-tuning is the single most impactful investment in this project. It converts a capable but generic model into a system that deeply understands JEE-specific visual vocabulary, output schemas, and Manim code patterns. Here is the complete approach from your existing PDFs to a deployed fine-tuned model.

6.1 Dataset Construction from Your PDFs

Your Sri Chaitanya and Narayana question papers with solution PDFs are the raw material. The construction pipeline is a one-time process:

1. Extract all question-solution pairs from PDFs using PyMuPDF (fitz). For each page, extract text and capture diagrams as cropped image regions. Each solution page maps to one question.
2. For questions with printed diagrams (Physics diagrams, chemical structures), save the diagram region as a separate PNG. The question image + solution text becomes one training example.
3. Use Claude API (one-time batch cost, not per-user) to convert each solution into the exact JSON scene format your Brain model should output. Review 10% manually for quality.
4. For each scene in the JSON, either: match it to your template library (label it as a template example), or write the correct Manim Scene code manually for novel cases. This builds your Coder training data.
5. Split: 80% train / 10% validation / 10% test. Hold the test set out completely until final evaluation.

Expected dataset sizes from your papers:

Questions per paper (estimate)	75 questions (25 per subject x 3 subjects)
Papers you appear to have	At minimum 2 (Sri Chaitanya GTM-07, Narayana GTM-06) — likely

	many more
Target training examples	500-800 minimum. 2000+ for best results.
Scenes per question (avg)	4-5 scenes. So 500 questions = ~2000-2500 scene examples.
Fine-tuning time (DGX A100)	4-8 hours for both models combined (overnight run)

6.2 Fine-Tuning the Brain (Qwen2.5-VL-7B)

This is the more important of the two fine-tuning jobs. The Brain's output quality determines everything downstream.

Framework: LLaMA-Factory or Axolotl — both support Qwen2.5-VL with LoRA

Method: LoRA (Low-Rank Adaptation) rank 32. Full fine-tune is not necessary and would require more VRAM.

Training format: Instruction-following format. Input: system prompt + question image + question text. Output: scene JSON.

```
CUDA_VISIBLE_DEVICES=7 llamafactory-cli train \
    --model_name_or_path Qwen/Qwen2.5-VL-7B-Instruct \
    --finetuning_type lora \
    --lora_rank 32 \
    --lora_target all \
    --dataset jee_brain_dataset \
    --template qwen2_vl \
    --output_dir ./checkpoints/brain_v1 \
    --num_train_epochs 3 \
    --per_device_train_batch_size 4 \
    --gradient_accumulation_steps 4 \
    --learning_rate 2e-4 \
    --bf16 true \
    --save_steps 100 \
    --eval_steps 50
```

After fine-tuning, load the LoRA adapter alongside the base model in vLLM. The adapter adds near-zero inference latency overhead while providing all the domain specialization benefits.

6.3 Fine-Tuning the Coder (Qwen2.5-Coder-7B)

The Coder fine-tuning teaches the model to produce syntactically correct, JEE-specific Manim code on the first attempt.

Training data format: Input: Director blueprint JSON + audio timestamps. Output: complete, runnable Manim Scene Python file.

Key constraint in training data: Every training example MUST be a Manim script that actually renders successfully. Never include broken code in training data — the model learns from what it sees.

Evaluation metric: first-pass render success rate. Measure on your held-out test set.
Target: >85% of test scenes render successfully without any retry.

6.4 Iterative Improvement Strategy

The fine-tuning is not a one-time event. As the system runs in production, you collect new training signal automatically:

- Every scene that required retries: the final successful code becomes a new training example
- Every new error pattern discovered: add it to the known-fix patcher library
- Every novel visual type you manually code for a question: add it to the template library
- Re-fine-tune monthly with accumulated data. Models improve continuously.

---

7. Implementation Tools & Infrastructure

7.1 Complete Tech Stack

Layer	Tool/Library	Version	Purpose	Runs On
LLM Serving	vLLM	0.6+	Inference server, continuous batching, prefix cache	DGX / RTX 4080
LLM Framework	LLaMA-Factory	latest	Fine-tuning pipeline for all models	DGX GPU 7
Orchestration	LangGraph	0.2+	Agentic loops, DAG execution, error handling	Windows PC
Task Queue	Celery + Redis	5.x	Job queue, worker management, rate limiting	Windows PC
API Layer	FastAPI	0.110+	HTTP endpoints, async request handling	Windows PC
Job State	PostgreSQL	16+	Job tracking, user data, cache metadata	Windows PC
Cache	Redis	7.x	Video URL cache, LaTeX cache, session state	Windows PC
Animation	Manim Community	0.18+	Core animation rendering	Windows PC
TTS	Kokoro-82M	latest	Text to speech, word timestamps	Windows PC (CPU)
Vector DB	ChromaDB	0.5+	RAG for Manim docs + validated examples	Windows PC
Embeddings	BGE-M3 (FlagEmbedding)	1.2+	Document embedding for ChromaDB	Windows PC (CPU)
Validation	Wolfram Alpha API	cloud	Math answer verification	Cloud API

Media	FFmpeg	6+	Audio mux, video concat, HLS generation	Windows PC
Sandbox	Docker (rootless)	25+	Isolated Manim execution	Windows PC
Storage	AWS S3 / Cloudflare R2	cloud	Video file hosting, CDN delivery	Cloud
Observability	LangSmith + Prometheus	cloud/local	LLM call tracing, infra metrics	Both
Fine-tune data	PyMuPDF + OpenCV	latest	PDF extraction, image cropping	Any

7.2 The Manim OpenGL Setup on Windows PC

Getting ManimGL (OpenGL renderer) running reliably on Windows is the most environment-sensitive part of the setup:

6. Install ManimCE via pip install manim. Then separately install manimgl for the OpenGL renderer.
7. Install a virtual framebuffer (Xvfb equivalent for Windows: use VcXsrv or configure WSL2 with display forwarding if running WSL).
8. Test render: manim --renderer=opengl -qh test_scene.py TestScene. If this produces output, OpenGL workers are working.
9. Create 6 persistent worker processes (use Supervisor or a Windows Service wrapper) that each import Manim at startup and stay alive waiting for Redis queue jobs.
10. LaTeX: Install MiKTeX (Windows). Pre-warm by compiling the 50 most common JEE expressions on first start.

7.3 LaTeX Cache Implementation

This is one of the highest ROI optimizations and takes about 2 hours to implement correctly:

```
LATEX_CACHE = {}  # or Redis hash for persistence across restarts

def get_or_compile_latex(expr: str) -> str:
    key = hashlib.sha256(expr.encode()).hexdigest()
    if key in LATEX_CACHE:
        return LATEX_CACHE[key]  # Return cached SVG path
    svg_path = manim_compile_latex(expr)  # Actual LaTeX call
    LATEX_CACHE[key] = svg_path
    return svg_path

# Monkey-patch into Manim's TeX class before rendering
# so all MathTex and Tex objects use the cache automatically
```

Pre-populate the cache with the 500 most common JEE expressions before handling any user requests. Extract these from your solution PDFs. This gives you a >70% cache hit rate from day one.

8. Modularity: DGX vs RTX 4080 Switching

The system uses a Backend Abstraction Layer — a thin wrapper around all LLM client calls that reads a single environment variable and routes to the appropriate inference endpoint:

```
# backend.py
import os
from openai import AsyncOpenAI

BACKENDS = {
    "dgx": {
        "brain": AsyncOpenAI(base_url="http://172.16.10.220:8001/v1",
api_key="EMPTY"),
        "coder": AsyncOpenAI(base_url="http://172.16.10.220:8002/v1",
api_key="EMPTY")
    },
    "local": {
        "brain": AsyncOpenAI(base_url="http://localhost:8001/v1",
api_key="EMPTY"),
        "coder": AsyncOpenAI(base_url="http://localhost:8002/v1",
api_key="EMPTY")
    }
}

ACTIVE = BACKENDS[os.getenv("INFERENCE_BACKEND", "local")]
brain_client = ACTIVE["brain"]
coder_client = ACTIVE["coder"]
```

Every part of the orchestrator imports brain_client and coder_client from this module. Switching from DGX to local is one command: export INFERENCE_BACKEND=local (or dgx). No other code changes. Both backends serve the same OpenAI-compatible API via vLLM.

This also makes it trivial to add a cloud API fallback (e.g. Together.ai hosts Qwen2.5-VL) for when both local options are unavailable.

---

9. Implementation Order & Milestones

The build order is designed so that every milestone produces something working and testable. Never build Phase N+1 until Phase N is demonstrably working on real JEE questions.

Milestone 1 — Coder + Render Loop (Week 1-2)

Goal: Given a hardcoded Manim blueprint, generate Python code, execute in Docker, fix errors, produce .mp4.

What you build: vLLM serving Coder-7B locally on RTX 4080, ChromaDB with Manim docs, LangGraph error loop, Docker sandbox, known-fix patcher.

Success criteria: 3 different hardcoded blueprints all produce valid .mp4 files, including recovering from at least one injected error.

Milestone 2 — Template Library (Week 2-3)

Goal: 10 fully working Manim templates covering the most common JEE visual types.

What you build: ProjectileTemplate, FreeBodyTemplate, AxesPlotTemplate, EquationTransformTemplate, LensRayTemplate, CircuitTemplate, ReactionMechanismTemplate, OrbitalDiagramTemplate, IntegrationAreaTemplate, GeometryConstructionTemplate.

Success criteria: Feed each template 5 different parameter sets from real JEE questions. All 50 renders succeed without any LLM call.

Milestone 3 — Brain Integration (Week 3-4)

Goal: Real JEE questions (text input only first, image later) produce correct scene JSON.

What you build: Brain prompt engineering, JSON schema enforcement in vLLM (grammar-based sampling), Wolfram Alpha validator integration.

Success criteria: 30 real questions from your papers produce scene JSON. Manually review: is the math correct? Is the visual_type sensible? Are narration strings pedagogically good?

Milestone 4 — Full Pipeline (Week 4-5)

Goal: End-to-end: question in, .mp4 out.

What you build: TTS integration (Kokoro), Director call, Template Matcher connecting to Coder, FFmpeg assembly, quality gate, HLS streaming endpoint.

Success criteria: 10 different JEE questions, submitted via API, produce watchable videos. Measure latency. Target: all under 3 minutes.

Milestone 5 — Fine-Tuning (Week 5-6)

Goal: Fine-tuned Brain and Coder models loaded and serving.

What you build: Dataset construction pipeline (PyMuPDF extraction + Claude API batch conversion), LLaMA-Factory fine-tuning config, evaluation harness.

Success criteria: Fine-tuned Brain produces scene JSON with >90% valid schema adherence on held-out test set. Fine-tuned Coder produces code with >85% first-pass render success.

Milestone 6 — Production Hardening (Week 6-7)

Goal: Cache, queue, observability, DGX/local switching, launch-ready.

What you build: Redis video cache with hash-based lookup, Celery job queue with priority lanes, LangSmith tracing integration, DGX backend config, LaTeX cache pre-population, load testing.

Success criteria: System handles 5 simultaneous requests gracefully (queue manages contention). DGX/local switch works. Cache hits return video in <1 second. System runs for

48 hours without manual intervention.

10. Complete Data Flow Summary

For reference, here is the complete data contract between every phase — what goes in and what comes out of each step:

Phase	Input	Output	Model/Tool	Avg Latency
0 - Cache	Question text/image	Cache hit URL OR job_id	Redis (SHA256/pHash)	<0.1s
1 - Brain	Question image + text	Scene JSON (schema v1)	Qwen2.5-VL-7B (ft)	4-6s
1.5 - Validator	final_answer string	VALID or RETRY signal	Wolfram Alpha API	1-2s
2A - TTS	Narration strings (all scenes)	.wav files + word timestamps	Kokoro-82M	0.3s
2B - Director	Scene JSON + durations	Coordinate blueprint JSON	Qwen2.5-VL-7B (ft)	2-4s
3 - Template	visual_type + visual_params	Manim .py file OR None	Python template lib	<0.1s
4 - Coder	Blueprint + RAG + timestamps	Manim .py file	Qwen2.5-Coder-7B (ft)	4-8s
4 - Render	Manim .py file	scene_N.mp4 OR traceback	Manim OpenGL + Docker	12-25s
4 - Error Fix	traceback + failed code	Patched .py OR LLM regen	Regex patcher + Coder	0-15s
5 - Quality Gate	scene_N.mp4 + audio duration	PASS or feedback signal	FFmpeg + Python	0.5s
6 - Assembly	All scene .mp4s + .wav files	final_output.mp4 + .m3u8	FFmpeg	3-5s
7 - Storage	final_output.mp4	CDN URL	S3/R2 + Redis cache write	2-4s

KEY DESIGN PRINCIPLES

The 5 decisions that make this system work at production quality

1. Templates first, LLM second. 70% of scenes never call the Coder LLM. Pre-written, validated templates are infinitely faster and more reliable than generated code.

2. Fine-tune for the domain. A 7B model fine-tuned on JEE content outperforms a generic 72B model on your specific task. Always. Do not skip fine-tuning.

3. Parallelise everything that can be. TTS and Director run simultaneously. All scenes render in parallel. Most of Phase 2 overlaps with Phase 1's tail. Every second saved in parallel is a real second off user latency.

4. LaTeX cache is the hidden multiplier. Most people don't know Manim's main bottleneck is LaTeX compilation, not rendering. The cache eliminates this. Pre-populate it before going live.

5. HLS streaming changes perceived latency from 60s to 25s. Start streaming scene 1 the instant it's ready. The user's brain treats it as a <30 second response even if total generation took 75 seconds.